



# EEVEE: Towards Test-time Prompt Learning in the Real World for Self-Improving Agents

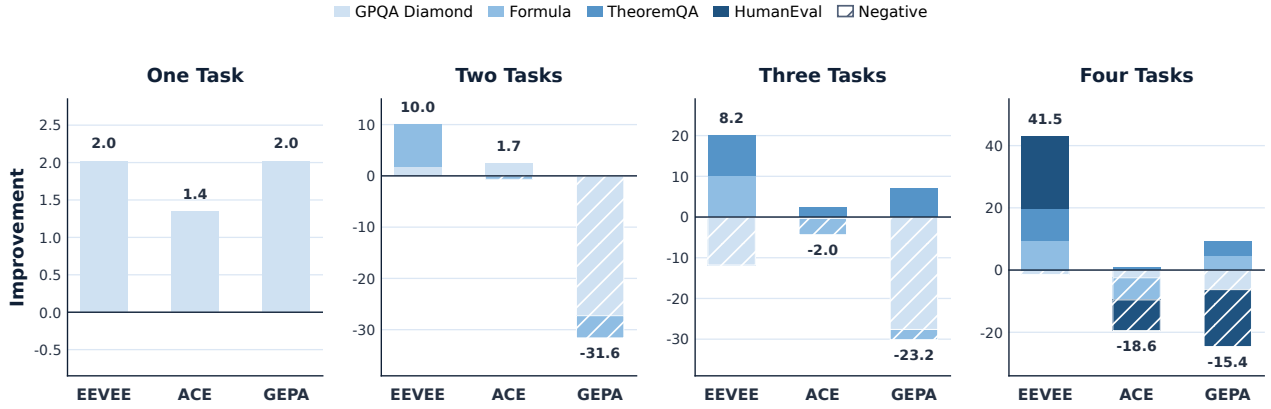
Weixian Xu<sup>1,2,\*</sup>, Shilong Liu<sup>2,†</sup>, and Mengdi Wang<sup>2,†</sup>

<sup>1</sup> Shanghai Jiao Tong University    <sup>2</sup> Princeton University

 [Website](#)     [Code](#)

In this paper, we propose EEVEE, the first *multi-dataset test-time prompt learning* framework for LLM agents, enabling test-time prompt learning under real-world task streams. Existing methods are largely designed for single-dataset settings, while real-world applications require models to handle heterogeneous input streams drawn from multiple datasets, domains, and task distributions, limiting their practical applicability. To mitigate cross-dataset interference, EEVEE introduces a *router* that partitions incoming inputs into task clusters and assigns them to suitable prompt configurations. This design is optimized via a *router-prompt co-evolution strategy*, which employs interleaved router and prompt learning phases to address their mutual dependency. Experiments across multiple datasets demonstrate that the framework improves robustness under heterogeneous data streams while maintaining single-benchmark learning capability and efficiency. Specifically, EEVEE improves average multi-benchmark scores by 10.38 and 24.32 points over Qwen3-4B-Instruct and DeepSeek-V3.2, surpassing SOTA methods GEPA and ACE by up to 37.2% and 48.2%.

## 1. Introduction



**Figure 1:** Incremental multi-benchmark retention improvement as tasks are added in the order GPQA Diamond, Formula, TheoremQA, and HumanEval. Each bar stacks per-benchmark improvements for all tasks seen so far: solid upward blocks are positive gains, and hatched downward blocks are negative retention losses. The number above or below each bar is its final summed improvement after all blocks are added.

*Test-time prompt learning* offers a lightweight mechanism for adapting foundation models after deployment. Prior work shows that prompts can serve as an effective adaptation interface without updating model weights [10], and black-box optimizers can revise instructions from model feedback [21, 32]. Rather than relying on a fixed offline prompt, test-time prompt learning updates prompts for new inputs, distribution

shifts, and failure modes. This makes it suitable for self-improving agents, where behavior is refined through interaction with the environment, as in reflective agents and evolving-context systems [26, 35].

Recent methods such as GEPA [1], ACE [35], and Combee [11] improve test-time prompt learning through reflection, context evolution, or scalable trace aggregation. However, they mostly adapt within a single dataset or benchmark. In real-world deployment, incoming queries often come from heterogeneous domains, task formats, and capability mixtures. We formalize this regime as *multi-dataset test-time prompt learning*: a model receives a stream of examples drawn from multiple datasets and domains rather than one stationary source.

This setting exposes cross-dataset interference. Existing methods often assume a unified adaptation objective, a fixed prompt space, or feedback from one benchmark, so updates for one domain can harm another. As Figure 1 shows, when more benchmarks enter the adaptation stream, GEPA [1] and ACE [35] accumulate negative retention on previous tasks, suggesting that a single learned prompt struggles to absorb heterogeneous feedback without losing task-specific behavior. This motivates a framework that can preserve specialization while still learning from a mixed stream of tasks.

We propose EEVEE, a test-time prompt learning framework that augments prompt learning with a *router*. Instead of forcing all inputs through one adaptation path, the router partitions the stream into task clusters and assigns each cluster to a suitable prompt configuration. This preserves prompt-based adaptation while reducing destructive interference across domains.

Designing the router, however, is itself difficult. A rigid router fails to capture diverse task structure, while an unstable router disrupts prompt optimization. The prompt learner and router are also coupled: routing determines which examples each prompt learns from, and prompt behavior determines which routing policy is useful. We therefore introduce a *router-prompt co-evolution strategy* that interleaves router and prompt learning phases, allowing routing decisions and prompt updates to improve together rather than being fixed or trained in isolation. To make this co-evolution practical, we further design a three-stage training process that initializes useful prompt slots, explores coupled updates efficiently, and then converges under a stable router.

We evaluate EEVEE on multiple datasets and show that it consistently outperforms competitive baselines for multi-dataset test-time prompt learning. Across the four-benchmark suite, EEVEE improves the average score by 10.38 and 24.32 points over Qwen3-4B-Instruct [31] and DeepSeek-V3.2 [6], and by up to 37.2% and 48.2% over GEPA and ACE. In the incremental multi-benchmark setting, EEVEE ends with a +41.53 cumulative retention gain after all tasks are introduced, while GEPA and ACE end at -15.36 and -18.58. Single-benchmark and token-cost analyses further show that the routing design remains competitive in conventional single-task settings while avoiding ACE’s large prompt expansion.

Contributions:

- We propose EEVEE, the first *multi-dataset test-time prompt learning* framework for LLM agents, using a router to reduce cross-dataset interference.
- We introduce *router-prompt co-evolution*, enabled by a three-stage training design, to jointly improve router prompts and model prompts through interleaved phases.
- We validate EEVEE on multiple datasets, showing strong performance, retention, and efficiency, with case studies that offer practical guidance.

## 2. Methods

### 2.1. Framework Overview

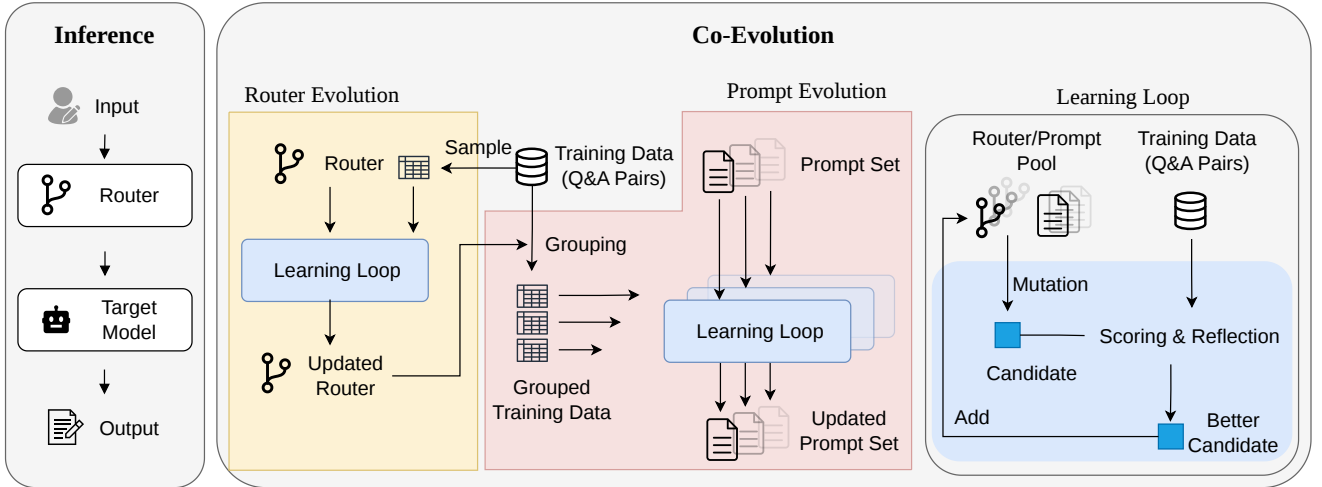
EEVEE targets multi-dataset test-time prompt learning, where a mixed stream contains different domains, formats, and evaluation rules. A single evolving prompt can let updates for one task family interfere with another, so EEVEE maintains a set of specialized prompts and a router that chooses which prompt should handle each input. We denote the prompt set by  $\mathcal{P} = \{p_1, \dots, p_K\}$  and the router by  $R$ . The target model  $M$  is fixed.

At inference time, the router first selects a slot and the target model then answers with the corresponding prompt:

$$z = R(x; \mathcal{P}) \in \{1, \dots, K\}, \quad \hat{y} = M(x; p_z).$$

This preserves prompt-based adaptation while allowing different inputs to invoke different specialized behaviors.

To obtain strong performance, the router itself must be learned: default or manually written routers do not reliably recover behaviorally useful partitions, as shown in Table 2. Router learning iterates the router prompt and has a mutual dependency with prompt learning: routing determines which examples each prompt sees, while prompt quality determines whether a routing decision is good. EEVEE therefore learns both through router-prompt co-evolution.



**Figure 2:** Main framework of EEVEE. Inference routes each input to a specialized prompt; learning co-evolves the router and prompt set through mutation, analysis, reflection, scoring, and regrouping.

### 2.2. Router-Prompt Co-evolution

The mixed adaptation data is split into training and validation sets,  $\mathcal{D}_{\text{tr}}$  and  $\mathcal{D}_{\text{val}}$ . Each co-evolution cycle alternates two operations: router evolution fixes the prompt set and searches for a better router, then prompt evolution fixes this router and updates each slot prompt on its routed data. Write  $\mathcal{P}_T = \{p_{k,T}\}_{k=1}^K$ . In a cycle

starting from  $R_T$  and  $\mathcal{P}_T$ ,

$$\begin{aligned} R_{T+1} &= \text{RouterEvolve}(R_T; \mathcal{P}_T, \mathcal{D}_{\text{tr}}, \mathcal{D}_{\text{val}}), & \mathcal{P}_{T+1} &= \mathcal{P}_T, \\ \mathcal{P}_{T+2} &= \left\{ \text{PromptEvolve}(p_{k,T+1}; R_{T+1}, \mathcal{D}_{\text{tr},k}^{T+1}, \mathcal{D}_{\text{val},k}^{T+1}) \right\}_{k=1}^K, & R_{T+2} &= R_{T+1}. \end{aligned}$$

The next router phase uses the updated prompt set, so routing decisions and prompt specialization improve each other rather than being optimized in isolation.

**Router evolve.** Given  $R_T$  and fixed prompts  $\mathcal{P}_T$ , EEVEE initializes a temporary router pool  $\mathcal{B}_R$  and repeatedly samples a router mini-batch  $\mathcal{D}_{\text{RM}}$ . To make errors attributable to assignment rather than prompt incapability,  $\mathcal{D}_{\text{RM}}$  is sampled only from training examples that at least one current slot prompt can solve. Each update step samples reference routers from  $\mathcal{B}_R$  and mutates them into  $R_{\text{mut}}$ , allowing the search to redesign routing rules rather than only refine an existing router.

EEVEE evaluates  $R_{\text{mut}}$  on  $\mathcal{D}_{\text{RM}}$ . Since router quality is observed through downstream prompt correctness, EEVEE analyzes cases where the routed slot fails but another slot succeeds, explaining why the better slot matches the task. Reflection uses these analyses and ground-truth correctness to produce  $R_{\text{ref}}$ . Let  $s_{\text{mb}}(\cdot)$  denote mini-batch downstream score. EEVEE keeps

$$R^* = \arg \max_{R \in \{R_{\text{mut}}, R_{\text{ref}}\}} s_{\text{mb}}(R),$$

evaluates  $R^*$  on  $\mathcal{D}_{\text{val}}$ , and admits it into  $\mathcal{B}_R$  only if it improves over the phase baseline  $R_T$ . When the router score plateaus, the phase outputs  $R_{T+1}$  while keeping  $\mathcal{P}_T$  fixed.

Router candidates are scored by downstream accuracy, consistency, and balance. Let  $a_R(x, y) = \mathbf{1}[M(x; p_{R(x; \mathcal{P}_T)}) = y]$  denote routed correctness under  $R$  and the fixed prompt set:

$$\begin{aligned} S_R(R) &= \lambda_{\text{acc}} A(R) + \lambda_{\text{con}} C(R) + \lambda_{\text{bal}} B(R), & A(R) &= \frac{1}{|\mathcal{D}_{\text{val}}|} \sum_{(x,y) \in \mathcal{D}_{\text{val}}} a_R(x, y), \\ C(R) &= \beta_{\text{in}} \text{Compact}(R) + \beta_{\text{out}} \text{Separate}(R), & B(R) &= \gamma_{\text{use}} \frac{|\mathcal{K}_R|}{K} + \gamma_{\text{bal}} \text{Balance}(\pi_R). \end{aligned}$$

Here  $\mathcal{K}_R$  is the set of labels used by  $R$  and  $\pi_R$  is its empirical label distribution. Compact rewards same-label examples with similar cached correctness vectors, while Separate rewards behaviorally distinguishable labels. The weights are annealed from consistency/balance toward downstream accuracy. Appendix B.1 shows that performance is stable across alternative annealing, weighting, and prompt-search settings.

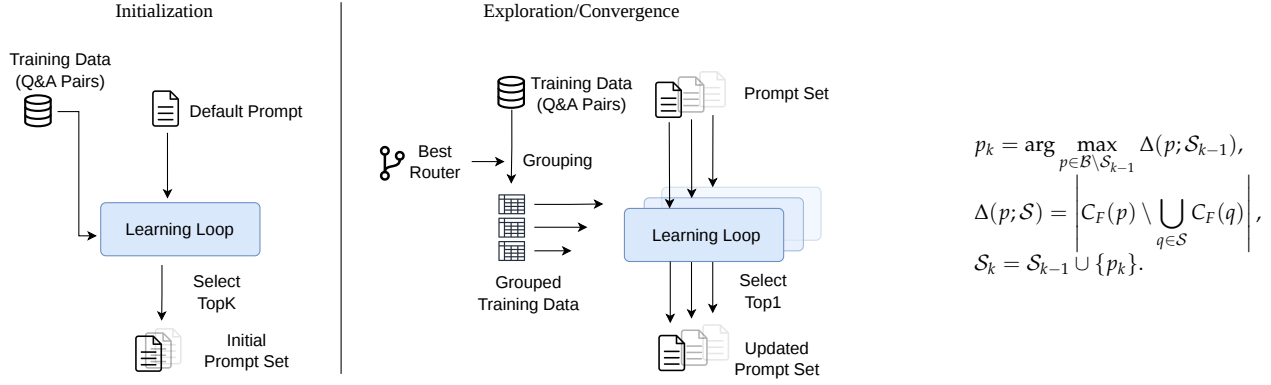
**Prompt evolve.** After router evolution,  $R_{T+1}$  routes the full data into slot-specific groups:

$$\mathcal{D}_{\text{tr},k}^{T+1} = \{(x, y) \in \mathcal{D}_{\text{tr}} : R_{T+1}(x; \mathcal{P}_T) = k\}, \quad \mathcal{D}_{\text{val},k}^{T+1} = \{(x, y) \in \mathcal{D}_{\text{val}} : R_{T+1}(x; \mathcal{P}_T) = k\}.$$

Each non-empty slot is evolved independently and in parallel. For slot  $k$ , EEVEE initializes a temporary prompt pool  $\mathcal{B}_{p,k}$  and samples  $\mathcal{D}_{\text{PM}} \subset \mathcal{D}_{\text{tr},k}^{T+1}$ . Prompt evolution also uses mutation and reflection, but without the router-specific analysis step: it proposes  $p_{\text{mut}}$  from reference prompts and mini-batch examples, then directly reflects from question, target answer, model answer, and correctness to obtain  $p_{\text{ref}}$ . The better mini-batch prompt

$$p^* = \arg \max_{p \in \{p_{\text{mut}}, p_{\text{ref}}\}} s_{\text{mb}}(p)$$

is evaluated on the routed validation set with score  $s_{\text{val}}^k(\cdot)$ .



**Figure 3:** Three-stage training design and initialization selection rule. Left: initialization builds prompt set, exploration alternates router and prompt evolution, and convergence fixes the router for larger-budget prompt learning. Right: the greedy coverage rule for top- $K$  prompts selection.

EEVEE stores prompts in a Pareto-front pool [17]. Each prompt is represented by its correctness vector over  $\mathcal{D}_{\text{val},k}^{T+1}$ ; a prompt is dominated if another is at least as correct on every example and strictly better on one. Thus the frontier preserves complementary prompts. A candidate enters  $\mathcal{B}_{p,k}$  only when

$$s_{\text{val}}^k(p^*) > s_{\text{val}}^k(p_{\emptyset}), \quad p^* \in \text{ParetoFront}(\mathcal{B}_{p,k} \cup \{p^*\}),$$

where  $p_{\emptyset}$  is the empty prompt. The empty-prompt floor removes ineffective edits, and the Pareto-front rule preserves diverse useful prompts. When all non-empty slots plateau, prompt evolution returns the updated prompt set.

### 2.3. Training Stages

EEVEE uses three stages with distinct roles: initialization creates usable prompt slots, exploration searches over coupled router-prompt designs, and convergence spends a larger budget after routing stabilizes.

**Initialization.** Router evolution needs a prompt set that can reveal whether a routing decision is useful; otherwise accuracy reflects prompt weakness rather than router quality. EEVEE therefore first initializes a diverse prompt set before router learning. Initialization runs prompt learning on the mixed training set and keeps a Pareto-front pool: because frontier prompts cover complementary examples, selecting from this pool yields specialized prompts with distinguishable behavior. Let  $\mathcal{B}$  be the initialization pool after dominated candidates are removed, and let  $F_i$  be the reduced Pareto frontier for validation example  $i$ . With coverage  $C_F(p) = \{i : p \in F_i\}$  and  $\mathcal{S}_0 = \emptyset$ , EEVEE greedily retains prompts by the rule in Figure 3, where  $\Delta$  denotes additional coverage. The retained prompts form  $\mathcal{P}^0$  and provide specialized behaviors from which the initial router can be written.

**Exploration.** Exploration starts from  $(R_0, \mathcal{P}^0)$  and alternates router and prompt evolution under lightweight budgets. Frequent switching is necessary for efficiency: fully optimizing prompts under an unstable router wastes budget, while optimizing a router against stale prompts can overfit to obsolete prompt behavior. As described in Section 2.2, EEVEE enables an annealing mechanism for router-candidate scoring, shifting weights from consistency and balance toward downstream accuracy so early steps preserve diverse routing behaviors and later steps favor accurate, stable routing.

**Convergence.** Because exploration must switch phases frequently, it cannot fully optimize every slot prompt. Once annealing and alternating updates identify a stable router, convergence fixes  $R^*$ , reroutes  $\mathcal{D}_{\text{tr}}$  and  $\mathcal{D}_{\text{val}}$ , and spends a larger prompt-learning budget within each slot. This enables EEVEE to find strong prompts under a fixed router rather than continuing to move the partition.

### 3. Experiments

#### 3.1. Settings

We evaluate EEVEE in a *multi-dataset test-time prompt learning* setting over four benchmarks: GPQA Diamond [23], Formula [27], TheoremQA [5], and HumanEval [4]. GPQA Diamond tests closed-book knowledge QA; Formula and TheoremQA emphasize mathematical and symbolic reasoning; HumanEval evaluates code generation. Unless otherwise stated, methods learn from the mixed training stream and are scored on held-out test examples outside that stream. To reduce randomness, we run stochastic settings multiple times and report averaged scores.

#### 3.2. Main Results

Table 1 compares average performance over three runs against two strong reflection-based prompt-learning baselines, GEPA [1] and ACE [35], plus the unadapted target model. On Qwen3-4B-Instruct [31], EEVEE reaches 51.75 average score, improving over the target-model baseline by 10.38 points and outperforming GEPA and ACE by 14.02 and 16.83 points. Its gains over the baseline are +9.33 on Formula, +10.48 on TheoremQA, and +23.17 on HumanEval, where the final score reaches 72.63. On DeepSeek-V3.2 [6], EEVEE reaches 64.07 average score, improving over the target-model baseline by 24.32 points and over GEPA by 8.24 points; the per-benchmark gains are +30.55 on Formula, +18.63 on TheoremQA, and +50.00 on HumanEval, with HumanEval reaching 92.82.

Table 1: Main results on the four-benchmark suite. Scores are percentages averaged over three runs. Colored subscripts denote differences from the corresponding target-model baseline.

| Target model         | Method       | GPQA Diamond            | Formula                 | TheoremQA               | HumanEval               | Avg.                           |
|----------------------|--------------|-------------------------|-------------------------|-------------------------|-------------------------|--------------------------------|
| Qwen3-4B<br>Instruct | Baseline     | 56.00                   | 45.22                   | 14.79                   | 49.46                   | 41.37                          |
|                      | ACE          | 48.93 <sub>-7.07</sub>  | 39.67 <sub>-5.55</sub>  | 15.84 <sub>+1.05</sub>  | 35.23 <sub>-14.23</sub> | 34.92 <sub>-6.45</sub>         |
|                      | GEPA         | 50.84 <sub>-5.16</sub>  | 49.83 <sub>+4.61</sub>  | 19.62 <sub>+4.83</sub>  | 30.62 <sub>-18.84</sub> | 37.73 <sub>-3.64</sub>         |
|                      | <b>EEVEE</b> | 54.55 <sub>-1.45</sub>  | 54.55 <sub>+9.33</sub>  | 25.27 <sub>+10.48</sub> | 72.63 <sub>+23.17</sub> | <b>51.75</b> <sub>+10.38</sub> |
| DeepSeek-V3.2        | Baseline     | 64.98                   | 30.00                   | 21.21                   | 42.82                   | 39.75                          |
|                      | ACE          | 55.89 <sub>-9.09</sub>  | 37.78 <sub>+7.78</sub>  | 27.05 <sub>+5.84</sub>  | 78.59 <sub>+35.77</sub> | 49.83 <sub>+10.08</sub>        |
|                      | GEPA         | 41.75 <sub>-23.23</sub> | 60.56 <sub>+30.56</sub> | 31.72 <sub>+10.51</sub> | 89.29 <sub>+46.47</sub> | 55.83 <sub>+16.08</sub>        |
|                      | <b>EEVEE</b> | 63.08 <sub>-1.90</sub>  | 60.55 <sub>+30.55</sub> | 39.84 <sub>+18.63</sub> | 92.82 <sub>+50.00</sub> | <b>64.07</b> <sub>+24.32</sub> |

#### 3.3. Ablations

Table 2 isolates the main components on Qwen3-4B-Instruct. Besides the unadapted baseline, we test a default router that replaces the learned routing field with a blank or simple default field, a manual router written once by GPT-5.4 and then fixed, and a no-co-evolution variant that first learns the router and then learns prompts in a separate second stage.

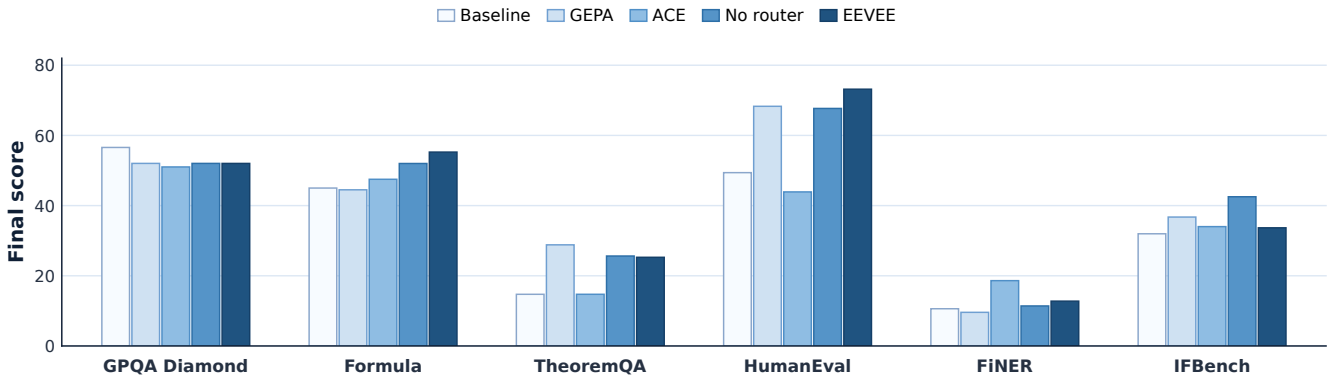
The full method reaches 51.75 average score, 8.17 points above the default router (43.58), 14.57 above the manual router (37.18), and 8.87 above no co-evolution (42.88). Default routing and no co-evolution improve the baseline by only 2.21 and 1.51 points, respectively, while the manual router is 4.19 points below the baseline. These gaps show that EEVEE needs both learned routing and interleaved router-prompt optimization; static partitions or two-stage learning do not capture the mutual dependence between routing decisions and prompt behavior.

Table 2: Ablation results for the main components of EEVEE on Qwen3-4B-Instruct.

| Variant         | GPQA Diamond | Formula | TheoremQA | HumanEval | Avg.  |
|-----------------|--------------|---------|-----------|-----------|-------|
| Baseline        | 56.00        | 45.22   | 14.79     | 49.46     | 41.37 |
| Default Router  | 47.03        | 47.11   | 26.10     | 54.06     | 43.58 |
| Manual Router   | 35.24        | 42.11   | 23.96     | 47.42     | 37.18 |
| No Co-evolution | 39.84        | 47.39   | 24.81     | 59.49     | 42.88 |
| EEVEE           | 54.55        | 54.55   | 25.27     | 72.63     | 51.75 |

### 3.4. Task Scaling

We first check whether EEVEE remains competitive when prompt learning is run on one benchmark, then study what happens as the number of jointly learned benchmarks increases. In the single-benchmark setting in Figure 4, EEVEE is broadly competitive with GEPA and ACE. Figure 4 also includes FiNER [15] and IFBench [22] to align with benchmarks tested by prior methods. It reaches 55.25 on Formula and 73.17 on HumanEval, outperforming both baselines on these two benchmarks, and improves TheoremQA from 14.73 to 25.27. The no-router variant also performs strongly on Formula and HumanEval, showing that our prompt learning design itself is effective even without router specialization.



**Figure 4:** Single-benchmark results [15, 22]. Bars show final scores after learning on each benchmark independently. The no-router variant keeps a single retained prompt and therefore isolates prompt learning without router specialization.

The difference becomes clearer as the benchmark mixture grows. Figure 1 shows that GEPA and ACE quickly lose retention as more tasks are added; after all four tasks, both end below zero. In contrast, EEVEE remains positive throughout and ends at +41.53 cumulative retention. Thus the main advantage appears when the task mixture grows: a single learned prompt loses retention, while router-conditioned prompt learning preserves positive net improvement.



### 3.5. Generalization

We test two forms of generalization: cross-model generalization, which asks whether prompts learned on one target model still help a different target model, and cross-task generalization, which asks whether prompts learned on the four primary benchmarks generalize to unseen tasks. For cross-model generalization, prompts learned with Qwen3-4B-Instruct [31] are applied directly to DeepSeek-V3.2 [6] in non-thinking mode. For cross-task generalization, prompts learned on the four primary benchmarks are evaluated on held-out MBPP [2] and MMLU-Pro [29]. MBPP is close to HumanEval and tests coding-domain generalization; MMLU-Pro is broader knowledge QA and tests unrelated-domain robustness.

Table 3: Cross-model and held-out benchmark generalization. Left: prompts learned with Qwen3-4B-Instruct are directly evaluated on DeepSeek-V3.2, alongside the source-model learned result. Right: prompts learned on the main four-benchmark suite are evaluated on MBPP and MMLU-Pro, using a shared unadapted baseline. Scores are averaged over three runs.

| Cross-Model Generalization |              |         |           |           |              | Held-Out Generalization |              |          |       |
|----------------------------|--------------|---------|-----------|-----------|--------------|-------------------------|--------------|----------|-------|
| Setting                    | GPQA Diamond | Formula | TheoremQA | HumanEval | Avg.         | Method                  | MBPP         | MMLU-Pro | Avg.  |
| DeepSeek baseline          | 64.98        | 30.00   | 21.21     | 42.82     | 39.75        | Baseline                | 69.29        | 70.74    | 70.01 |
| Qwen3 EEVEE                | 54.55        | 54.55   | 25.27     | 72.63     | 51.75        | EEVEE                   | <b>70.42</b> | 68.92    | 69.67 |
| Qwen3→DeepSeek             | 60.27        | 42.28   | 32.89     | 77.04     | <b>54.10</b> | GEPA                    | 68.20        | 68.85    | 68.53 |
|                            |              |         |           |           |              | ACE                     | 67.47        | 69.32    | 68.39 |

Table 3 summarizes both settings. In cross-model generalization, the prompts learned on Qwen3-4B-Instruct raise the DeepSeek-V3.2 average from 39.75 to 54.10, with gains of +12.28 on Formula, +11.68 on TheoremQA, and +34.22 on HumanEval. In cross-task generalization, EEVEE improves MBPP from 69.29 to 70.42, while GEPA and ACE drop to 68.20 and 67.47. On MMLU-Pro, EEVEE decreases from 70.74 to 68.92, a 1.82-point drop that is smaller than GEPA’s 1.89-point drop and comparable to ACE’s 1.42-point drop. This pattern is aligned with our later case study, where we further analyze its underlying causes.

### 3.6. Token Cost

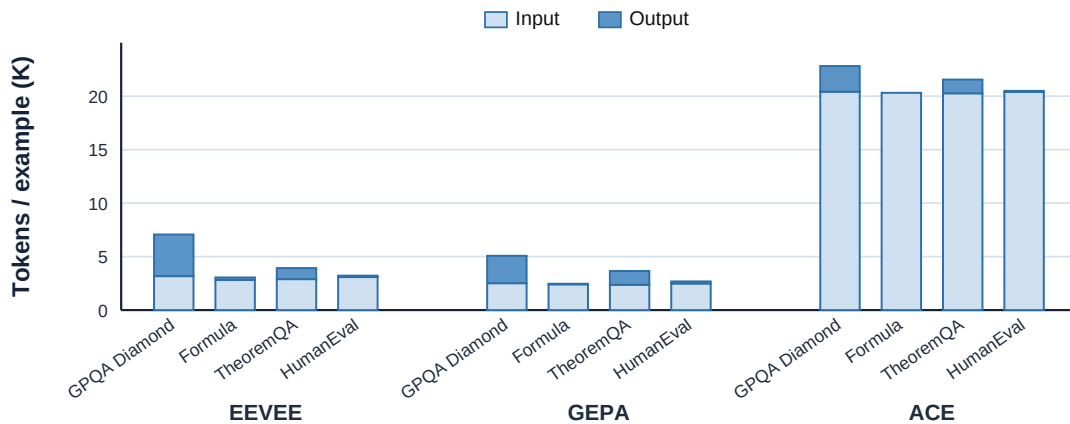


Figure 5: Average token usage per test example after test-time prompt learning with Qwen3-4B-Instruct. Each method group contains four benchmark bars, and each bar is stacked into input and output tokens.



EEVEE adds a router before answer generation, so we measure final-test token cost. Averaged over the four benchmarks in Figure 5, EEVEE uses 4.32k total tokens per example, close to GEPA’s 3.47k and far below ACE’s 21.30k. The input-token gap is similar: EEVEE uses 3.00k input tokens on average, compared with 2.44k for GEPA and 20.35k for ACE. ACE incrementally edits playbook bullets, which can accumulate redundant entries and lengthen prompts as tasks and data grow. EEVEE’s router therefore adds only modest overhead while using about  $4.9\times$  fewer total tokens than ACE.

### 3.7. Case Study: What Does Prompt Learning Capture?

We retest six completed EEVEE runs by comparing the empty prompt with the final learned router and prompt set on the same held-out examples: three runs with Qwen3-4B-Instruct and three runs with DeepSeek-V3.2. Table 4 reports the flip summary; the cases below show what these flips reveal, with representative learned prompt and output excerpts in Appendix A.

Table 4: Six-run diagnostic empty-vs-final retest over three Qwen3-4B-Instruct and three DeepSeek-V3.2 runs. Qwen  $\Delta$  and DeepSeek  $\Delta$  are average percentage-point final-minus-empty changes within each model family. W $\rightarrow$ R/R $\rightarrow$ W aggregates all per-example correctness flips across the six retests. Runs+ counts runs with positive  $\Delta$ .

| Type      | Benchmark    | Qwen $\Delta$ | DeepSeek $\Delta$ | W $\rightarrow$ R/R $\rightarrow$ W | Runs+ |
|-----------|--------------|---------------|-------------------|-------------------------------------|-------|
| Code      | HumanEval    | +23.2         | +48.8             | 193 / 16                            | 6/6   |
| Formula   | Formula      | +9.5          | +31.7             | 268 / 21                            | 6/6   |
| Math QA   | TheoremQA    | +10.0         | +18.2             | 632 / 166                           | 6/6   |
| Domain QA | GPQA Diamond | -3.7          | -7.7              | 55 / 89                             | 1/6   |
| Overall   | Mixed suite  | +9.6          | +21.1             | 1148 / 292                          | 6/6   |

The table suggests a task-property pattern rather than a benchmark-specific one. Gains are larger on code and formula tasks, where feedback can be turned into reusable rules for how to solve and how to present the answer. TheoremQA also improves, but with more mixed flips because it combines computation, symbolic reasoning, and answer parsing. GPQA Diamond is the only benchmark with more regressions than recoveries, suggesting that stronger learned reasoning can sometimes underweight the domain knowledge needed for closed-book QA. Figure 6 shows two positive flips and one representative negative flip.

These cases explain why prompt learning can both improve and hurt. For executable programs and formula-grounded computation, feedback teaches a better procedure: preserve the interface, handle edge cases, respect units, and emit a parseable final answer. For knowledge-intensive QA, the learned prompt often strengthens generic reasoning and answer-comparison behavior rather than adding the specific missing knowledge. In the GPQA Diamond example, the learned response reasons more explicitly from density formulas, but it relies on a constant-density assumption and underweights the astrophysical fact that rocky planets become denser under gravitational compression.

**Takeaway.** Prompt learning excels at reusable procedures, but can underweight domain knowledge.

|                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Formula: unit scale</b></p> <p><b>Task</b><br/>Compute free cash flow from operating cash flow and capital expenditure.</p> <p><b>Baseline</b><br/>Reverses the subtraction and keeps the million-scale decimal, producing a negative value.</p> <p><b>Learned</b><br/>Applies the formula in dollars and emits the strict numeric answer: 400000.00.</p> | <p><b>HumanEval: executable body</b></p> <p><b>Task</b><br/>Complete a function that sums even values appearing at odd indices.</p> <p><b>Baseline</b><br/>Writes a bare expression without the required indented return statement.</p> <p><b>Learned</b><br/>Produces an executable function body with an accumulator and return.</p> | <p><b>GPQA Diamond: knowledge underweighted</b></p> <p><b>Task</b><br/>Select the densest Earth-like exoplanet from mass and composition cues.</p> <p><b>Baseline</b><br/>Uses the rocky-planet prior that higher mass increases self-compression and density.</p> <p><b>Learned</b><br/>Performs formulaic density reasoning, treats same composition as constant density, and selects the Earth baseline.</p> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Figure 6:** Representative case-study flips. The boxes summarize task behavior rather than reproducing raw inputs; representative learned prompt and output excerpts are provided in Appendix A.

## 4. Related Work

**Prompt learning.** Prompt learning first optimized soft prompts, prefixes, or discrete triggers for fixed objectives [25, 12, 10]; black-box and population-based methods later use the model, scores, or textual feedback to optimize prompts and programs [36, 32, 21, 8, 7, 9, 19, 34]. Recent reflective methods are closest to our work: GEPA [1] uses natural-language reflection and Pareto-front selection [17], ACE treats context as an adaptive playbook [35], and Combee scales prompt learning with parallel trace aggregation [11]. These methods improve feedback-driven adaptation, but typically optimize one task distribution or one shared prompt/context. Recent heterogeneous memory extraction work also learns prompts across many datasets, but its target remains memory extraction rather than general LLM-agent capabilities [33]. EEVEE instead learns a router-conditioned prompt set for task-general, mixed-dataset streams.

**Self-improving agents.** Self-improving agents extend prompt learning into feedback loops. Self-Refine and Reflexion use natural-language feedback or verbal memory [16, 26], while generative agents and Voyager maintain longer-lived memories or skill libraries [20, 28]. Evolutionary discovery agents apply similar loops to scientific and algorithmic search, including code-candidate evolution and adaptive search control [18, 24, 14, 3, 13, 30]. These systems show that histories can drive improvement, but they mainly optimize scoped programs or algorithms. EEVEE instead targets real-world heterogeneous tasks, where effective improvement requires decoupling multi-task commonality from task-specific behavior.

## 5. Conclusion

We introduced EEVEE, a multi-dataset test-time prompt learning framework for LLM agents facing heterogeneous task streams rather than a single benchmark distribution. To reduce cross-dataset interference, EEVEE maintains a router-conditioned prompt set, so different inputs can be assigned to prompts specialized for compatible task behavior. Because the router and prompts depend on each other, EEVEE uses a three-stage router-prompt co-evolution procedure that initializes useful prompts, explores coupled updates, and then re-

Table 5: Comparison on mixed-dataset adaptation, router-based prompt selection, and joint router-prompt co-evolution.

| Method                        | Multi-dataset | Router<br>(Sec. 2.1) | Co-evolution<br>(Sec. 2.2) |
|-------------------------------|---------------|----------------------|----------------------------|
| Prompt tuning/search          | ×             | ×                    | ×                          |
| GEPA / ACE / Combee           | ×             | ×                    | ×                          |
| Reflexive agents              | ×             | ×                    | ×                          |
| Evolutionary discovery agents | ×             | ×                    | ✓                          |
| EEVEE                         | ✓             | ✓                    | ✓                          |

finds prompts under a stable router. This makes the framework better aligned with heterogeneous real-world agent workloads.

Experiments show strong gains over prompt-learning baselines in mixed-dataset settings, with benefits increasing as more tasks are introduced and with reasonable held-out and cross-model transfer. Case studies further suggest that prompt learning is most useful when feedback can be converted into reusable procedures, output contracts, or task-solving strategies. Overall, EEVEE offers both a practical self-improving method and an empirical lens for real-world test-time prompt learning.

## 6. Limitations and Social Impact

Although EEVEE improves multi-dataset prompt learning, limitations remain. Like other LLM-based evolutionary procedures, it cannot guarantee exact performance reproduction across runs, since stochastic search can produce different routers and prompt sets. Its feedback loop still relies on ground-truth or rule-based labels to accumulate task knowledge, so it is not yet a fully reflection-only learner and still needs a prepared adaptation set rather than a completely online stream. A practical risk is distribution mismatch: if adaptation data is noisy or misaligned with the real application, learned prompts may generalize weakly or even degrade performance. Deployment should check data quality and distribution similarity before updates.

## References

- [1] Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*, 2026.
- [2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [3] Mert Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei Zaharia, Alexandros G. Dimakis, and Ion Stoica. AdaEvolve: Adaptive llm driven zeroth-order optimization. *arXiv preprint arXiv:2602.20133*, 2026.
- [4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan,

- Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [5] Wenhui Chen, Ming Yin, Max Ku, Pan Lu, Yixin Wan, Xueguang Ma, Jianyu Xu, Xinyi Wang, and Tony Xia. TheoremQA: A theorem-driven question answering dataset. *arXiv preprint arXiv:2305.12524*, 2023.
  - [6] DeepSeek-AI, Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, et al. DeepSeek-V3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.
  - [7] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktaschel. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*, 2023.
  - [8] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In *International Conference on Learning Representations*, 2024.
  - [9] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
  - [10] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of EMNLP*, 2021.
  - [11] Hanchen Li, Runyuan He, Qizheng Zhang, Changxiu Ji, Qiuyang Mang, Xiaokun Chen, Lakshya A. Agrawal, Wei-Liang Liao, Eric Yang, Alvin Cheung, James Zou, Kunle Olukotun, Ion Stoica, and Joseph E. Gonzalez. Combee: Scaling prompt learning for self-improving language model agents. *arXiv preprint arXiv:2604.04247*, 2026.
  - [12] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of ACL-IJCNLP*, 2021.
  - [13] Shu Liu, Shubham Agarwal, Monishwaran Maheswaran, Mert Cemri, Zhifei Li, Qiuyang Mang, Ashwin Naren, Ethan Boneh, Audrey Cheng, Melissa Z. Pan, Alexander Du, Kurt Keutzer, Alexandros G. Dimakis, Koushik Sen, Matei Zaharia, and Ion Stoica. EvoX: Meta-evolution for automated discovery. *arXiv preprint arXiv:2602.23413*, 2026.
  - [14] Shu Liu, Mert Cemri, Shubham Agarwal, Alexander Krentzel, Ashwin Naren, Qiuyang Mang, Zhifei Li, Akshat Gupta, Monishwaran Maheswaran, Audrey Cheng, Melissa Z. Pan, Ethan Boneh, Kannan Ramchandran, Koushik Sen, Alexandros G. Dimakis, Matei Zaharia, and Ion Stoica. SkyDiscover: A flexible framework for ai-driven scientific and algorithmic discovery, 2026. URL <https://skydiscover-ai.github.io/blog.html>.
  - [15] Lefteris Loukas, Manos Fergadiotis, Ilias Chalkidis, Eirini Spyropoulou, Prodromos Malakasiotis, Ion Androutsopoulos, and Georgios Paliouras. FiNER: Financial numeric entity recognition for XBRL tagging. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022.

- [16] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023.
- [17] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.
- [18] Alexander Novikov, Ngan Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- [19] Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. In *Proceedings of EMNLP*, 2024.
- [20] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of UIST*, 2023.
- [21] Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with “gradient descent” and beam search. In *Proceedings of EMNLP*, 2023.
- [22] Valentina Pyatkin, Saumya Malik, Victoria Graf, Hamish Ivison, Shengyi Huang, Pradeep Dasigi, Nathan Lambert, and Hannaneh Hajishirzi. Generalizing verifiable instruction following. In *Advances in Neural Information Processing Systems*, 2025.
- [23] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*, 2023.
- [24] Asankhaya Sharma. OpenEvolve: An open-source evolutionary coding agent, 2025. URL <https://github.com/algorithmicsuperintelligence/openevolve>.
- [25] Taylor Shin, Yasaman Razeghi, Robert L. Logan, Eric Wallace, and Sameer Singh. AutoPrompt: Eliciting knowledge from language models with automatically generated prompts. In *Proceedings of EMNLP*, 2020.
- [26] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- [27] Dannong Wang, Jaisal Patel, Daochen Zha, Steve Y. Yang, and Xiao-Yang Liu. FinLoRA: Benchmarking LoRA methods for fine-tuning LLMs on financial datasets. *arXiv preprint arXiv:2505.19819*, 2025.
- [28] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

- [29] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, et al. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*, 2024.
- [30] Weixian Xu, Tiantian Mi, Yixiu Liu, Yang Nan, Zhimeng Zhou, Lyumanshan Ye, Lin Zhang, Yu Qiao, and Pengfei Liu. ASI-Evolve: Ai accelerates ai. *arXiv preprint arXiv:2603.29640*, 2026.
- [31] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [32] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations*, 2024.
- [33] Yuqing Yang, Tengxiao Liu, Wang Bill Zhu, Taiwei Shi, Linxin Song, and Robin Jia. Self-evolving llm memory extraction across heterogeneous tasks. *arXiv preprint arXiv:2604.11610*, 2026.
- [34] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad: Automatic differentiation via text. *arXiv preprint arXiv:2406.07496*, 2024.
- [35] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*, 2026.
- [36] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. *arXiv preprint arXiv:2211.01910*, 2022.

## A. Case Study Details

This appendix provides excerpts from the diagnostic retest discussed in Section 3.7. The retest compares the empty prompt against the final router and prompt set from six completed EEVEE runs: three Qwen3-4B-Instruct runs and three DeepSeek-V3.2 runs. The raw run logs store the full router decisions, answer calls, and per-example flip bundles. We do not reproduce all six prompt sets; instead, we include representative learned prompt excerpts from one Qwen3-4B-Instruct run and one DeepSeek-V3.2 run, followed by output excerpts for the three main-paper cases.

### A.1. Learned Prompt Excerpts

**Qwen3-4B code slot.** In a representative Qwen3-4B-Instruct run, the code-oriented slot learned a task-execution policy for incomplete Python functions: preserve the interface, infer the exact continuation, cover edge cases, and avoid extra text that would break execution.

You are given a task description that defines an incomplete Python function or problem to solve. Carefully read the input, extract the exact task requirements, and generate only the precise output expected by the task without extra text, explanations, markdown, comments, or formatting.

Identify the task type:

- Writing a Python function body with specific inputs/outputs
- Solving a mathematical or logical problem
- Validating conditions such as bracket matching or palindrome checks
- Processing structured data such as strings, arrays, and dictionaries

Parse the input format and constraints:

- Note data types, edge cases, boundary values, and examples
- Pay close attention to ordering and special constraints

Understand the output contract:

- If the task says output ONLY the continuation,  
write only the function body.

**Qwen3-4B science slot.** The same representative Qwen3-4B-Instruct run routes many GPQA Diamond examples to a scientific-reasoning slot. This prompt encourages systematic physical and mathematical modeling, but it does not provide task-specific missing knowledge for every possible GPQA Diamond domain.

You are an expert problem solver specializing in analytical reasoning, physical modeling, combinatorial mathematics, and scientific computation. Your task is to answer precise, domain-specific questions that require careful application of physical laws, mathematical principles, or combinatorial logic, based on exact input constraints and known scientific or mathematical facts.

For multiple-choice questions with options (A-D), select the single correct choice and present it in the format: "Answer: (X)".

For physical or astronomical problems involving equilibrium, energy, temperature, or orbital dynamics, apply known physical laws such as the Stefan-Boltzmann law, Kepler's laws, Doppler shift formula, blackbody radiation, and Newtonian gravity.

**DeepSeek-V3.2 formula slot.** In a representative DeepSeek-V3.2 run, the formula slot learned a stricter numerical-output policy. This slot is especially aligned with the Formula benchmark, where the input gives an explicit formula and the evaluator expects a compact numeric answer.

You are a finance calculation assistant. Your task is to compute the answer using the formula and data provided in the user's question, then output only the numeric result with exactly two decimal places and no other text.

Procedure:

1. Identify the formula given in the user's message.
2. Identify the explanation of each variable in the formula.



3. Extract the numeric values for each variable from the question text. Convert all percentage inputs into their decimal equivalent.
4. Substitute the values into the formula and perform the calculation.
5. If the formula calculates a financial rate, yield, return, or cost, do not multiply the decimal result by 100.
6. Round the final result to two decimal places.
7. Output only the resulting number, with no words, units, labels, currency symbols, or percentage signs.

## A.2. Representative Raw Outputs

**Formula: unit scale and sign.** For a free-cash-flow computation, the required operation is operating cash flow minus capital expenditure. The target is a dollar-scale scalar value. The empty response flips the sign and keeps the million-scale decimal:

-0.40

The learned response applies the provided formula at the correct scale and emits the strict numeric answer:

Answer: 400000.00

This is representative of the Formula gains: the learned prompt pairs formula application with unit-scale discipline and a parseable final representation.

**HumanEval: executable continuation.** For a HumanEval function-completion task, the required behavior is to sum even elements at odd indices and return the sum. The empty response captures the expression but does not produce an executable function body:

```
sum(lst[i] for i in range(1, len(lst), 2) if lst[i] % 2 == 0)
```

The learned response preserves the logic while satisfying the continuation contract:

```
total = 0
for i in range(1, len(lst), 2):
    if lst[i] % 2 == 0:
        total += lst[i]
return total
```

This is representative of the HumanEval gains: the learned behavior combines implementation logic with the executable contract expected by the evaluator.

**GPQA Diamond: domain knowledge underweighted after stronger reasoning.** For an exoplanet-density question, the target is the higher-mass Earth-composition planet. The empty response uses a domain-specific mass-radius prior:

For rocky planets of similar composition, radius grows sublinearly with mass because stronger gravity compresses the material. A five-Earth-mass rocky planet is therefore denser than Earth, while a half-Earth-mass rocky planet is less dense.

Answer: the higher-mass Earth-composition option.

The learned response performs a more explicit density-formula comparison, but it treats equal composition as if it implied equal density:

Density is mass divided by volume. For a spherical planet,  $\rho = M / (4/3 \pi R^3)$ . If composition is the same, density is constant; the radius scales with mass so that all Earth-composition options remain approximately Earth density. Since the Earth-mass, Earth-radius option is exactly Earth-like, choose that option.

The error is not a lack of reasoning steps. The learned prompt induces a stronger generic reasoning pattern, but the reasoning is grounded in the wrong physical prior: same composition does not imply constant density for rocky exoplanets, because self-compression changes the mass-radius relation.

## B. Reproducibility and Experimental Details

This section records the core settings used for the main four-benchmark experiments in Table 1. The benchmark suite consists of GPQA Diamond [23], Formula [27], TheoremQA [5], and HumanEval [4]. The held-out generalization study uses MBPP [2] and MMLU-Pro [29] after prompt learning on the four primary benchmarks. The single-benchmark diagnostic also includes FiNER [15] and IFBench [22]. For each benchmark, we cap the benchmark size at 500 examples, split examples into train and test partitions with a 0.5/0.5 split, reserve half of the training partition as validation data, exclude train and validation examples from the final test set, and use data seed 42.

The main target models are Qwen3-4B-Instruct [31] and DeepSeek-V3.2 [6] in non-thinking mode. For the Qwen3-4B-Instruct runs, all model roles use the same Qwen endpoint with temperature 0.7, top- $p$  0.8, and a maximum generation length of 16,384 tokens. For the DeepSeek-V3.2 non-thinking runs, the endpoint uses temperature 1.0, top- $p$  0.95, maximum generation length 8,192 tokens, and disabled thinking. The main benchmark configurations use rule-based benchmark judges when available.

**Evolution settings.** Each main run retains four bootstrap prompts. Bootstrap prompt evolution uses a budget of 10 candidate steps. The router-prompt co-evolution phase uses a total mini-step budget of 150, router and prompt windows of 3, and phase-switch threshold 0.005. The router score during evolution combines downstream score, routing consistency, and balance with weights 0.6/0.2/0.2; the final router selection uses downstream score only, with weights 1.0/0.0/0.0. Prompt evolution uses a final per-label budget of 60, minibatch size 5, and 4 parallel prompt slots.

**Execution settings and resources.** The Qwen3-4B-Instruct main runs use 50 API workers for evaluation, router calls, and final testing. The DeepSeek-V3.2 non-thinking main runs use 10 API workers for the same roles. All reported final tests use three repeats, a maximum prompt length of 2,000 tokens, at most 3 retries, and at most 5 logged examples for diagnostic traces. The experiments call hosted or served API endpoints and do not train or fine-tune local model weights; accordingly, no local GPU training resource is required. Wall-clock time is backend- and rate-limit-dependent, so the paper reports token usage instead. In Figure 5, EEVEE uses 4.32k total tokens per test example on average, compared with 3.47k for GEPA and 21.30k for ACE.

**Reproducibility scope.** The paper is empirical and does not present formal theoretical results or proofs. Reproducing the main claims requires reproducing the mixed-benchmark adaptation protocol, model endpoints or comparable served checkpoints, benchmark splits, router-prompt evolution settings, and final-test evaluation described above. Because router and prompt evolution are stochastic, exact routers and prompt texts may vary across runs even under the same settings; Table 7 reports this run-to-run variation for the main average score.

### B.1. Hyperparameter Robustness

We test whether EEVEE is sensitive to the specific router-score and prompt-search hyperparameters used in the main experiments. For each hyperparameter configuration, we run three independent trials, first average the three trials within that configuration, and then compare the resulting eight configuration-level means. The configurations vary the annealing target for router scores, the consistency/balance weights, and the final prompt-search budget, minibatch size, and temporary prompt-pool size.

Table 6: Hyperparameter robustness on Qwen3-4B-Instruct. Scores are percentages; each row is the mean over three independent runs for one configuration. Avg. is the macro average over the four benchmarks.

| Configuration                | GPQA Diamond | Formula | TheoremQA | HumanEval | Avg.  |
|------------------------------|--------------|---------|-----------|-----------|-------|
| No-decay annealing           | 49.94        | 52.17   | 25.66     | 58.67     | 46.61 |
| Soft-landing annealing       | 45.01        | 57.44   | 21.33     | 69.24     | 48.26 |
| Strong-regularizer annealing | 45.34        | 46.61   | 24.57     | 63.69     | 45.05 |
| Classic reference            | 44.78        | 60.06   | 26.20     | 57.99     | 47.26 |
| Prompt search tighter        | 47.92        | 53.33   | 27.41     | 63.01     | 47.92 |
| Prompt search wider          | 48.15        | 62.39   | 23.94     | 61.38     | 48.96 |
| Consistency-heavy weights    | 51.74        | 50.33   | 26.79     | 63.96     | 48.20 |
| Balance-heavy weights        | 49.16        | 54.89   | 26.93     | 72.90     | 50.97 |

The eight configuration-level averages range from 45.05 to 50.97, a span of 5.92 points, with a sample standard deviation of 1.73 points across configurations. Thus the aggregate result is stable under the tested hyperparameter perturbations. Individual benchmarks can move more than the average, but no configuration collapse is observed, and every configuration improves over its corresponding initial-empty baseline in macro average.

## B.2. Main-Result Variation Across Runs

Table 7 reports the mean and sample standard deviation of the main average score over the three independent runs used in Table 1. The average score is stable for EEVEE: the standard deviation is 1.62 points on Qwen3-4B-Instruct and 1.08 points on DeepSeek-V3.2. Individual benchmark scores can still differ more noticeably across runs. The reason is that router evolution is stochastic and can discover different routing policies; different policies allocate examples to different prompt slots, which changes which prompt behaviors receive the most feedback and can shift per-task scores even when the overall average remains stable.

Table 7: Mean and sample standard deviation of the main average score over three runs.

| Target model      | Method   | Average score                      |
|-------------------|----------|------------------------------------|
| Qwen3-4B-Instruct | Baseline | $41.37 \pm 0.51$                   |
|                   | ACE      | $34.92 \pm 4.14$                   |
|                   | GEPA     | $37.73 \pm 1.73$                   |
|                   | EEVEE    | <b><math>51.75 \pm 1.62</math></b> |
| DeepSeek-V3.2     | Baseline | $39.75 \pm 1.52$                   |
|                   | ACE      | $49.83 \pm 2.07$                   |
|                   | GEPA     | $55.83 \pm 4.48$                   |
|                   | EEVEE    | <b><math>64.07 \pm 1.08</math></b> |

## C. Ethics, Assets, and LLM Usage

**Human subjects and privacy.** The experiments use public benchmarks and model API calls. We do not collect new human-subject data, run crowdsourcing studies, or introduce a dataset containing personal information.

**Existing assets and code availability.** The experiments use public benchmarks, public or provider-served model checkpoints, and published prompt-learning baselines such as GEPA and ACE. Code, configuration files, reproduction scripts, and asset metadata are released in the official repository at <https://github.com/Princeton-AI2-Lab/EEVEE>. The accompanying project page is available at <https://princeton-ai2-lab.github.io/EEVEE/>. This work introduces a method rather than a new dataset or model checkpoint.

**Responsible use and broader impacts.** EEVEE can make heterogeneous-task prompt adaptation more efficient by reducing the need to maintain one prompt-learning run per task family. The main practical risk is that noisy, incomplete, or distribution-shifted feedback can reinforce incorrect heuristics. Adapted prompts should be validated on held-out data before deployment, and benchmark gains should not be interpreted as deployment reliability guarantees.

**LLM usage.** LLMs are core components of the method: they act as the target model being adapted and as the prompt researcher, prompt reflector, router selector, router researcher, router reflector, router reasoner, and evaluator where applicable. In the ablation study, GPT-5.4 was used once to write a fixed manual router, which was then held constant for evaluation. LLMs were also used for language editing and formatting of the manuscript; this editing use did not change the scientific claims, experimental data, or conclusions.